PROJECT WORK DOCUMENT

AI Outbound Follow-Up Calling System

with Dashboard UI, Auto-Rescheduling & Data Logging

Technical Specification & Scope of Work

Document Version	1.0
Prepared For	Assigned Developer
Document Type	Scope of Work (SOW) / Technical Specification
Project Type	MVP — Production-Ready
Primary Integration	VAPI (Voice AI Platform)
Target Volume	200–500 outbound calls/day
Primary Language	English
Use Case	Automated Client Follow-Up Calls

Table of Contents

1. **Executive Summary**
2. **Project Objectives**
3. **Scope of Work**
4. **Functional Requirements**
   - 4.1 Client Management Module
   - 4.2 AI Outbound Calling Engine
   - 4.3 Conversation Flow Design (Follow-Up Use Case)
   - 4.4 Automatic Rescheduling Module
   - 4.5 Conversation Data Capture & Summarization
   - 4.6 Excel / Google Sheets Logging
5. **User Interface (UI) Requirements**
6. **Technical Requirements**
   - 6.1 Technology Stack — Developer's Choice
   - 6.2 High-Level System Architecture
   - 6.3 Minimum Database Schema
   - 6.4 Backend API — Minimum Endpoints
   - 6.5 Capacity, Performance & Concurrency
7. **Non-Functional Requirements**
8. **Testing Requirements**
9. **Deliverables**
10. **Acceptance Criteria**
11. **Assumptions & Client Responsibilities**
12. **Acknowledgement & Sign-Off**

1. Executive Summary

This document defines the complete scope of work, functional requirements, technical specifications, and acceptance criteria for the development of an AI-powered Outbound Follow-Up Calling System.

The system will autonomously place outbound calls to clients retrieved from a managed database, conduct natural English-language voice conversations using VAPI's AI voice agents, capture client responses, handle rescheduling requests automatically, and log every interaction into structured reports. A web-based dashboard will allow operators to manage clients, monitor call activity, and download call summaries.

This is an MVP (Minimum Viable Product) but must be built to production standards — reliable, observable, and ready for daily operational use at a volume of 200 to 500 calls per day.

1.1 At a Glance

Area	Summary
Primary Goal	Automate client follow-up calls end-to-end without manual dialing
Voice Platform	VAPI (Voice AI Platform) — required
Language	English only
Call Volume	200–500 outbound calls/day
Concurrency Target	Up to 10 concurrent calls during peak hours
User Interface	Web-based admin dashboard (required)
Tech Stack	Developer's choice — must justify selection
Output Format	Excel/Google Sheets summary reports per call
Deployment	Cloud-hosted, production-grade

2. Project Objectives

The system must achieve the following objectives:

1. Automate outbound follow-up calls to clients retrieved from a managed client database, with zero manual dialing.
2. Deliver natural, human-like conversations in English using VAPI's AI voice agents, personalized with each client's specific data and follow-up context.
3. Capture and structure client responses during every call, producing a clean summary of the interaction.
4. Handle rescheduling requests automatically — if a client asks to be called at a later time, the system must book that slot and place the call automatically at the requested time.
5. Provide a complete operator dashboard (UI) for client management, call scheduling, live monitoring, and reporting.
6. Log every call outcome into a structured Excel/Google Sheets format for downstream review and reporting.
7. Operate reliably at a daily volume of 200–500 calls with observable uptime and error handling.

3. Scope of Work

3.1 In Scope

- End-to-end outbound calling system built on top of VAPI.
- Telephony integration (VAPI handles the underlying carrier — e.g., Twilio — and the AI voice agent).
- Full-featured admin dashboard with client CRUD, CSV import, call scheduling, live status, and reporting.
- Conversation flow design for the client follow-up use case.
- Automatic rescheduling logic with a persistent call queue.
- Post-call summarization (AI-generated) and structured data capture.
- Excel (.xlsx) export and Google Sheets sync of call logs.
- Database design, backend APIs, frontend UI, authentication, and cloud deployment.
- Documentation: setup guide, environment configuration, API reference, user manual.
- Testing: unit tests on critical paths, integration tests with VAPI sandbox, end-to-end test with real phone numbers.

3.2 Out of Scope (This Phase)

- Inbound call handling (the system is outbound-only in Phase 1).
- SMS, WhatsApp, or email outreach channels.
- CRM integrations (Salesforce, HubSpot, etc.) — deferred to Phase 2.
- Multi-language voice support — English only.
- Mobile native apps — the dashboard is web-only (desktop-first, responsive acceptable).
- Advanced role-based access control — a single admin role is acceptable for MVP.
- Telephony carrier accounts — the client will provide VAPI and Twilio credentials.

4. Functional Requirements

4.1 Client Management Module

The system must allow operators to manage the client database through the dashboard.

4.1.1 Required Capabilities

- Add a single client manually through a form.
- Bulk import clients via CSV/Excel upload with column mapping and validation preview.
- View paginated, searchable, filterable client list.
- Edit any client record.
- Delete individual or multiple clients (with confirmation).
- Mark clients as active/inactive (inactive clients are skipped by the calling engine).

4.1.2 Client Data Fields (Minimum Schema)

Field	Type	Required	Notes
client_id	UUID	Yes	Auto-generated primary key
full_name	String	Yes	Used in conversation personalization
phone_number	String (E.164)	Yes	Validated on save; format +[country][number]
email	String	No	Optional
follow_up_context	Text	Yes	What the follow-up is about — fed into AI prompt
previous_interaction	Text	No	Summary of last call/interaction
scheduled_call_time	DateTime	Yes	When the system should call
timezone	String	Yes	Client's timezone (e.g., Asia/Karachi)
status	Enum	Yes	pending / in_progress / completed / failed / rescheduled
notes	Text	No	Operator notes
created_at / updated_at	DateTime	Yes	Timestamps

4.2 AI Outbound Calling Engine

The core engine that places outbound calls via VAPI and runs the AI conversation.

4.2.1 Calling Engine Requirements

- Integrate with VAPI using their official API/SDK.
- Maintain a persistent call queue with status tracking per client.
- A background scheduler must poll the queue and trigger calls at their scheduled time (respecting the client's timezone).
- Support concurrent outbound calls — target up to 10 simultaneous calls at peak hours.
- Retry logic for failed calls: on no-answer/busy/voicemail, retry up to 3 times with configurable delays (e.g., +30 min, +2 hr, next day same time).
- Detect voicemail/answering-machine and handle it per configuration (leave message or hang up).
- Capture call duration, disposition, and recording URL for every attempt.
- Respect calling-window restrictions (e.g., only call between 9am–8pm client local time) — configurable per deployment.

4.2.2 VAPI Integration Requirements

- Configure a VAPI Assistant for the follow-up use case with a system prompt that includes client-specific context at runtime (dynamic variables).
- Select an appropriate English voice model (e.g., ElevenLabs, PlayHT, or VAPI-native voice) — finalize with the client during kickoff.
- Handle VAPI webhooks: call-started, call-ended, message events, function-calls, and transcript events.
- Use VAPI's function-calling capability so the AI can trigger actions mid-call (e.g., book-reschedule, mark-not-interested, confirm-details).
- Store all VAPI call IDs against our internal call records for auditability.

4.3 Conversation Flow Design (Follow-Up Use Case)

The AI agent's conversation must follow a consistent, natural flow for follow-up calls. The developer is responsible for crafting the initial VAPI system prompt; the flow below is the required minimum behavior.

4.3.1 Required Conversation Stages

8. Greeting & Identity Verification — The AI introduces itself by the configured business name and confirms it is speaking to the correct client by name.
9. Purpose Statement — The AI clearly states the reason for the follow-up call, referencing the follow_up_context field from the client record.
10. Permission to Proceed — The AI asks if it's a good time to talk. If no, it offers to reschedule (see 4.4).
11. Information Gathering — The AI asks the follow-up questions as per the conversation script, capturing structured answers.
12. Summary & Confirmation — The AI repeats key answers back to the client for confirmation.
13. Reschedule or Next Action — If needed, the AI books a reschedule or confirms the next step.

14. Polite Closure — The AI thanks the client and ends the call.

4.3.2 Guardrails

- The AI must never claim to be a human if asked directly.
- The AI must stay on-topic — no open-ended chit-chat beyond the follow-up purpose.
- The AI must handle interruptions gracefully (client talking over it).
- Maximum call length — hard cutoff at 10 minutes per call.
- If the client requests a human, the AI must log this and end the call politely.

4.4 Automatic Rescheduling Module

A critical feature: if a client requests to be called at a different time, the system must handle it entirely without operator intervention.

4.4.1 Reschedule Flow

15. The AI detects a reschedule request (via function-calling or intent detection).
16. The AI asks for the preferred date and time.
17. The AI confirms the new slot back to the client.
18. The backend parses the date/time (with timezone), validates it, and persists the updated scheduled_call_time.
19. The client's status is set to 'rescheduled'.
20. The scheduler will pick up the client again at the new time and call automatically.

4.4.2 Reschedule Rules

- Maximum 3 reschedules per client (configurable). After that, the client is marked as 'manual-follow-up-required' for operator review.
- Rescheduled times must fall within the configured calling window. If outside, the AI should offer the next nearest valid slot.
- All reschedule events must be logged with original time, new time, and reason.

4.5 Conversation Data Capture & Summarization

Every call must produce a structured record of what happened.

4.5.1 Captured Per Call

- Full conversation transcript (from VAPI).
- AI-generated summary of the call (3–5 sentences).
- Structured answers to the follow-up questions (JSON/key-value).
- Call disposition: completed, no-answer, voicemail, refused, rescheduled, failed.
- Call duration, start/end timestamps.
- Recording URL (if recording is enabled in VAPI).
- Sentiment tag (positive / neutral / negative) — derived from the transcript.

4.5.2 Summarization Approach

After a call ends, the backend should call an LLM (e.g., OpenAI GPT-4 class or the VAPI-provided end-of-call report) with the transcript and a structured extraction prompt. Output fields must match the Excel schema in 4.6.

4.6 Excel / Google Sheets Logging

Every completed call must be written to a persistent, downloadable spreadsheet log.

4.6.1 Minimum Columns

Column	Description
Call ID	Internal unique ID
Client Name	From client record
Phone Number	E.164 format
Call Start Time	Timestamp in UTC + client TZ
Call End Time	Timestamp
Duration (sec)	Total duration
Call Status	Final disposition
Attempt Number	1st, 2nd, 3rd, etc.
AI Summary	3–5 sentence summary
Key Answers (JSON)	Structured extracted answers
Sentiment	positive / neutral / negative
Rescheduled To	New datetime if rescheduled, else blank
Recording URL	Link to call recording
Transcript URL	Link to full transcript
Operator Notes	Editable field in UI

4.6.2 Export Requirements

- The dashboard must offer a 'Download Excel' button with date-range and status filters.
- A Google Sheets sync option must also be available — the operator links a Google Sheet once, and every new call is appended to that sheet in real time.
- Both exports use the same schema.

5. User Interface (UI) Requirements

A clean, professional, and responsive web-based dashboard is required. It is the primary tool operators will use daily, so usability matters. Desktop-first is acceptable; mobile responsiveness is nice-to-have for MVP.

5.1 Authentication

- Email + password login.
- Secure session management (JWT or server sessions).
- A single 'Admin' role is sufficient for MVP.
- Password reset flow via email (nice-to-have for MVP).

5.2 Required Pages / Screens

5.2.1 Dashboard (Home)

- Summary tiles: Calls Today, Successful, Failed, Rescheduled, Pending.
- Chart: call volume and outcomes over the last 7 / 30 days.
- Live activity feed: recently completed calls with status.

5.2.2 Clients Page

- Paginated table with columns: Name, Phone, Scheduled Time, Status, Last Action.
- Search bar (by name/phone) and filters (by status).
- 'Add Client' button → modal form.
- 'Import CSV' button → upload + preview + confirm.
- Row actions: View, Edit, Delete, Call Now, Reschedule.

5.2.3 Call Scheduling Page

- Calendar or list view of upcoming scheduled calls.
- Drag-and-drop or quick-edit to move a call to a new time (MVP: simple edit modal is acceptable).
- Bulk actions: pause/resume the calling engine globally.

5.2.4 Call Logs Page

- Paginated table of all past calls.
- Columns: Client, Phone, Start Time, Duration, Status, Sentiment, Summary (truncated with hover).
- Filters: date range, status, sentiment, client.
- Click a row → Call Detail view (below).

5.2.5 Call Detail View

- Full transcript (scrollable).

- AI summary and structured answers.
- Audio player for the recording.
- Timeline: call events (dial, pickup, AI turns, user turns, reschedule event, end).
- Operator notes (editable).

5.2.6 Reports / Export Page

- Date-range selector and filters.
- 'Download Excel' and 'Sync to Google Sheets' buttons.
- Preview of the data before download.

5.2.7 Settings Page

- VAPI API key management (masked input).
- Business name, caller ID number (configured in VAPI/Twilio).
- Calling window (start time, end time, timezone).
- Retry policy configuration.
- Conversation script / VAPI system prompt — editable textarea with a 'Test Prompt' button.
- Google Sheets integration connection.

5.3 UI Design Standards

- Clean, modern, professional — comparable to Linear, Vercel, or Intercom dashboards in polish.
- Consistent design tokens (colors, spacing, typography).
- All destructive actions (delete, stop calling) must have a confirmation step.
- Loading and empty states must be handled explicitly — no blank screens.
- Error states must surface backend errors clearly to the operator.
- Accessible: proper contrast ratios, keyboard navigation, form labels.

6. Technical Requirements

6.1 Technology Stack — Developer's Choice

The developer is free to select the technology stack, but the selection must be justified in writing and approved by the client before implementation begins. The stack must meet the constraints below.

6.1.1 Mandatory Constraints

- Voice layer: VAPI (non-negotiable).
- Backend: must be a modern, actively maintained runtime (e.g., Node.js, Python, Go) — no legacy PHP monoliths.
- Frontend: must be a component-based framework (React, Next.js, Vue, Svelte).
- Database: relational (PostgreSQL strongly preferred) for transactional data; document DB acceptable only with justification.
- Hosting: production-grade cloud (AWS, GCP, Azure, Railway, Render, Fly.io — developer may propose).
- Version control: Git, hosted on GitHub (private repo provided by client).

6.1.2 Recommended Reference Stack

The following is a known-good reference stack the developer may adopt directly:

Layer	Recommended Technology
Frontend	Next.js 14+ (App Router) + Tailwind CSS + shadcn/ui
Backend	Node.js + Fastify or Express, OR Python + FastAPI
Database	PostgreSQL (managed — Supabase, Neon, or RDS)
Job Queue	BullMQ (Node) or Celery (Python) with Redis
Voice AI	VAPI
Telephony	Twilio (configured inside VAPI)
LLM (summarization)	OpenAI GPT-4o-mini or GPT-4o
Auth	Auth.js / Clerk / Supabase Auth
Hosting (frontend)	Vercel
Hosting (backend + workers)	Railway, Render, or Fly.io
Monitoring	Sentry (errors) + a simple uptime ping
Logging	Structured JSON logs + a log aggregator (Logtail, Axiom, or cloud-native)

6.2 High-Level System Architecture

The system must be built as the following logical components:

21. Web Dashboard (Frontend) — talks only to the Backend API.
22. Backend API — serves the dashboard, authenticates users, manages client records, exposes reporting endpoints, and receives VAPI webhooks.
23. Scheduler Worker — a background job that continuously polls the call queue and triggers calls whose scheduled_call_time has arrived (respecting timezone and calling window).
24. Call Orchestration Worker — initiates outbound calls via VAPI, listens for VAPI webhooks, updates call state, triggers retries.
25. Post-Call Processor — on call-ended, fetches transcript, generates summary via LLM, extracts structured answers, writes to DB and exports to Sheets/Excel.
26. Database — PostgreSQL with tables for clients, calls, call_events, reschedules, users, settings.
27. Message Queue (Redis) — decouples the scheduler from the orchestration worker and supports retries and concurrency.

6.3 Minimum Database Schema

6.3.1 clients

See Section 4.1.2 for fields.

6.3.2 calls

Field	Type	Notes
call_id	UUID	Primary key
client_id	UUID (FK)	References clients
vapi_call_id	String	External ID from VAPI
attempt_number	Integer	1, 2, 3...
started_at	DateTime	
ended_at	DateTime	
duration_seconds	Integer	
status	Enum	completed / no_answer / voicemail / failed / rescheduled / refused
transcript	Text	Full transcript
summary	Text	AI-generated
structured_answers	JSONB	Extracted key-value answers

Field	Type	Notes
sentiment	Enum	positive / neutral / negative
recording_url	String	
created_at	DateTime	

6.3.3 reschedules

Field	Type	Notes
reschedule_id	UUID	Primary key
client_id	UUID (FK)	
call_id	UUID (FK)	Call during which reschedule was requested
original_time	DateTime	
new_time	DateTime	
reason	Text	Client's stated reason, if any
created_at	DateTime	

6.4 Backend API — Minimum Endpoints

REST or RPC is acceptable. The following are the minimum endpoints the developer must implement:

Method	Endpoint	Purpose
POST	/auth/login	Authenticate user
POST	/auth/logout	End session
GET	/clients	List with pagination + filters
POST	/clients	Create single client
POST	/clients/import	Bulk CSV import
GET	/clients/:id	Retrieve one
PATCH	/clients/:id	Update
DELETE	/clients/:id	Delete
POST	/clients/:id/call-now	Trigger an immediate call
GET	/calls	List with filters

Method	Endpoint	Purpose
GET	/calls/:id	Full call detail with transcript
GET	/reports/export	Excel download (with filters)
POST	/integrations/google-sheets/connect	OAuth-connect a sheet
POST	/webhooks/vapi	VAPI event receiver (public, signed)
GET	/settings	Read settings
PATCH	/settings	Update settings
GET	/dashboard/stats	Home-page metrics

6.5 Capacity, Performance & Concurrency

- Daily volume: 200–500 calls.
- Peak concurrency: design for 10 simultaneous outbound calls. The calling engine must not serialize calls.
- Average call duration assumption: 2–3 minutes. Budget: 500 calls × 3 min ≈ 25 call-hours/day — easily handled by VAPI + Twilio at modest concurrency.
- Dashboard response time: < 500 ms p95 for list endpoints.
- Webhook processing: VAPI webhook handler must respond < 200 ms and push heavy work to the queue.
- Scheduler tick: every 30 seconds (configurable).

7. Non-Functional Requirements

7.1 Security

- All secrets (VAPI API key, Twilio credentials, database URL, OpenAI key) must be stored in environment variables or a secrets manager — never in code.
- The .env file must never be committed to Git. A .env.example must be provided instead.
- HTTPS everywhere — no plain HTTP in production.
- Passwords stored using bcrypt/argon2, never plaintext.
- VAPI webhooks must be signature-verified.
- Input validation on every API endpoint (zod, pydantic, or equivalent).
- Rate limiting on auth and public endpoints.
- Basic audit log: who created/edited/deleted clients and when.

7.2 Reliability

- The calling engine must survive process restarts without dropping queued calls.
- Failed VAPI calls must be retried per the retry policy.
- Database backups: daily automated backups (managed DB providers handle this — confirm enabled).
- Graceful degradation: if the LLM summarization service is down, the call is still logged with raw transcript; summarization retries later.

7.3 Observability

- Structured JSON logging for all backend events.
- Error tracking via Sentry or equivalent.
- An uptime endpoint (/health) monitored by an external service.
- Key metrics visible in the dashboard: calls initiated, calls completed, average duration, failure rate.

7.4 Data Retention & Privacy

- Call recordings and transcripts retained for a configurable period (default: 90 days).
- Operator must be able to delete a client's data on request (GDPR-style delete).
- Only authenticated operators may view client data or call recordings.

7.5 Maintainability

- Clean, commented, readable code.
- Consistent code style enforced by a linter + formatter (ESLint+Prettier, Ruff, etc.).
- README in the repo with local setup steps runnable in < 15 minutes.
- Environment-based configuration (dev / staging / production).

8. Testing Requirements

8.1 Required Test Coverage

- Unit tests for all business logic in the scheduler, retry engine, and reschedule parser.
- Integration tests for VAPI webhook handlers using fixture payloads.
- End-to-end test: a real test phone number is called via VAPI, completes a short scripted conversation, and the full log appears in the dashboard and in the Excel export.
- Manual QA script covering every UI page and action.

8.2 Test Phase Deliverables

- A test CSV with 5–10 fake clients (real test phone numbers).
- Live demo: place 3 sample calls (completed, rescheduled, failed) and walk through the dashboard and exported sheet.
- Bug-fix cycle of at least one round after UAT (User Acceptance Testing).

9. Deliverables

Upon completion, the developer must deliver the following:

#	Deliverable	Format
1	Source code — full repo with clean commit history	Private GitHub repo
2	Deployed production environment (frontend + backend + DB + workers)	Live URL
3	Deployed staging environment	Live URL
4	Environment configuration guide	.env.example + README
5	Admin user manual — how operators use the dashboard	PDF or Notion doc
6	Technical documentation — architecture, data model, API reference	Markdown in repo
7	VAPI assistant configuration (exported JSON or documented in repo)	In repo
8	Demo/UAT session (live walkthrough)	Video call
9	Handover session: code walkthrough with the client's future team	Video call + recording
10	30-day post-launch bug-fix support	Agreed SLA

10. Acceptance Criteria

The project is considered complete when every item below is demonstrably true:

28. An operator can log in, add 10 test clients manually, and import 20 more via CSV.
29. The scheduler automatically picks up a client when their scheduled_call_time arrives and initiates a call via VAPI.
30. The AI conducts a coherent follow-up conversation in English, personalized with the client's follow_up_context.
31. When a test client asks to reschedule, the system books the new time correctly and calls back automatically at that time.
32. Every completed call produces a transcript, AI summary, structured answers, and sentiment — visible in the dashboard within 60 seconds of call end.
33. The Excel download and Google Sheets sync contain every required column and reflect the most recent calls.
34. The system sustains 10 concurrent calls in a load test without errors or dropped webhooks.
35. Failed calls are automatically retried per the configured retry policy.
36. All secrets are in environment variables; no credentials are in the repo.
37. Error tracking (Sentry) is active; one intentional error appears in the dashboard within 5 minutes.
38. Documentation and handover are complete; a new engineer can run the system locally in under 15 minutes using only the README.

11. Assumptions & Client Responsibilities

11.1 Client Provides

- VAPI account and API key with sufficient credits.
- Twilio account (or whichever carrier VAPI is configured with) with a purchased outbound number and adequate balance.
- OpenAI API key (for post-call summarization) with sufficient credits.
- Domain name(s) for the dashboard.
- Google Cloud project + OAuth credentials for the Google Sheets integration.
- Private GitHub repo with developer access.
- Hosting account (Vercel, Railway, Render, etc.) or budget to create one.
- Final approval of the VAPI conversation script before production use.

11.2 Assumptions

- English is the only voice language for Phase 1.
- All clients being called have provided prior consent to receive calls (legal compliance is the client's responsibility, not the developer's).
- Call volumes will remain within 200–500/day; scaling beyond this is Phase 2.
- VAPI's service availability is a third-party dependency and outside developer control.

12. Acknowledgement & Sign-Off

By signing below, the developer confirms they have read and understood this document in full, agree to the scope and acceptance criteria defined herein, and commit to delivering the project accordingly.

Developer Name: _____	Client Name: _____
Signature: _____	Signature: _____
Date: _____	Date: _____

— End of Document —